

OOP 历年卷考点与易错点总结

资料范围：当前文件夹内 2005-2006.pdf、2007-2008.pdf、2008-2009.pdf、2016-2017.pdf、2018-2019.pdf、2018-2019-有答案.pdf、2019-2020.pdf、2023-2024.pdf、23-24_wk_midterm.pdf。其中早年卷多为扫描版，本总结按可读题面和重复题型归纳，重点面向 C++ 语法题、输出题、程序填空题和函数题。

老师划重点版

老师明确给出的知识点：class、inheritance、casting、polymorphism、pass by reference、static member variable、template、exception、virtual、operator overloading、constructor and destructor。题型包括程序输出题、选择题、填空题、函数/类设计题；模板类必须复习。

题型	可能考法	必会知识点	易错点	复习动作
程序输出题	给一段类/继承/重载代码，写每一行输出	构造析构、虚函数、函数隐藏、引用返回、异常流	忘记临时对象和局部对象析构；把拷贝构造看成赋值；忽略 endl	按“构造 -> 调用 -> 析构”逐行模拟
选择题	判断概念、选正确/错误说法	class 默认 private、struct 默认 public、virtual、static、operator、new/delete	概念题容易被绝对化表述骗，如 “all members” “always” “only”	背专题速查表，遇到绝对词重点审
填空题	补函数声明、返回类型、模板参数、类成员	引用、const、模板、友元、运算符重载	返回类型漏 &；模板类名漏 <T>；成员函数类外定义漏作用域	先看调用点，再反推签名
函数/类设计题	按 judge 代码补 class 或模板类	接口匹配、构造函数、运算符、虚函数、多态删除	自己改 judge；函数名/参数 const 不匹配；输出格式多空格/少空格	从 main 和测试函数逐行列 “需要哪些接口”

老师点名知识点	重点掌握	常见考题信号	一句话规则
class	访问控制、封装、friend、const 成员函数	private 成员能否访问、同类对象互访	类成员函数可以访问同类任意对象的 private
inheritance	public 继承、多继承、对象切片、构造析构顺序	class B : public A、A a = b	派生对象构造先基类后派生，析构反过来
casting	static_cast、dynamic_cast、转换构造、类型转换函数	基类指针转派生、operator B()、static_cast<double>	dynamic_cast 用于多态类型，失败指针返回 nullptr
polymorphism	基类指针/引用调用虚函数	Base *p = new Derived; p->f();	virtual 调用看动态类型，非 virtual 看静态类型
pass by reference	引用传参、引用返回、避免拷贝	int& f(int&)、A& operator=	引用是别名，修改引用就是修改原对象

老师点名知识点	重点掌握	常见考题信号	一句话规则
<code>static member variable</code>	类内声明、类外定义、对象共享	<code>static int m; int A::m;</code>	static 数据成员不属于某个对象，所有对象共享
<code>template</code>	函数模板、类模板、特化、模板类	<code>template<class T></code> 、 <code>Array<T></code> 、 <code>CQueue<T></code>	模板类类外定义必须写 <code>template<class T></code> 和 <code>Class<T>::</code>
<code>exception</code>	<code>throw</code> 、 <code>catch</code> 、重抛、catch 顺序	<code>catch(...)</code> 、 <code>throw;</code>	抛出后跳到匹配 catch，try 后续语句不执行
<code>virtual</code>	虚函数、纯虚函数、虚析构	<code>virtual void f()</code> 、 <code>=0</code> 、 <code>delete Base*</code>	多态基类析构函数应为 virtual
<code>operator overloading</code>	<code>[]</code> 、 <code>++</code> 、 <code>=</code> 、 <code><<</code> ，类型转换	<code>operator[]</code> 、 <code>operator++(int)</code>	<code>[]</code> 、 <code>()</code> 、 <code>-></code> 、 <code>=</code> 必须是成员函数
<code>constructor and destructor</code>	默认构造、拷贝构造、析构顺序	<code>A a; A b=a; delete p;</code>	析构函数不能重载，构造函数不能 virtual

高频考点总表

模块	常见题型	核心规则	易错点	典型判断方法
构造函数	默认构造、带参构造、拷贝构造、 <code>explicit</code>	只要用户声明了任意构造函数，编译器不再自动生成默认构造函数； <code>explicit</code> 禁止拷贝初始化	<code>C c = 7;</code> 不能调用 <code>explicit C(int)</code> ，可能转而调用 <code>C(double)</code>	区分 <code>C c(7)</code> 直接初始化和 <code>C c = 7</code> 拷贝初始化
拷贝构造	<code>A a2 = b;</code> 、函数传参、返回对象	拷贝构造一般写 <code>A(const A&)</code>	写成 <code>A(A&)</code> 不能绑定 <code>const</code> 对象和临时对象	看到同类对象初始化新对象，就考虑拷贝构造
赋值运算符	<code>a2 = a1;</code>	赋值发生在两个已经存在的对象之间，通常返回 <code>A&</code>	把 <code>A a = b;</code> 误判成赋值；实际上是拷贝构造	“是否已经构造完成”是判断构造/赋值的关键
析构函数	输出顺序、 <code>delete basePtr</code>	析构顺序与构造相反；派生类析构先执行，再执行基类析构	基类析构非 <code>virtual</code> 时用基类指针删除派生对象可能未定义行为	先列对象生命周期，再按作用域退出和 <code>delete</code> 顺序写输出
派生类构造	初始化基类子对象	派生类应在初始化列表中显式调用基类构造函数，如 <code>B(int x): A(x) {}</code> ；若不写，编译器尝试调用 <code>A()</code>	不能在派生类初始化列表里直接初始化基类成员；基类无默认构造时不写 <code>A(args)</code> 会报错	构造顺序永远是：基类 -> 派生类成员 -> 派生类构造函数体
派生类拷贝构造	拷贝基类子对象和成员对象	自定义 <code>B(const B& b)</code> 时，应在初始化列表写： <code>A(b)</code> ， <code>a(b.a)</code> ， <code>i(b.i)</code>	如果不写，基类子对象和成员对象会默认构造，不会自动拷贝构造	函数体执行前，所有基类和成员已经构造完
编译器生成的派生类拷贝构造	没有自定义 <code>B(const B&)</code> 时	编译器生成的拷贝构造会自动拷贝基类子对象和成员对象	和“自定义但没写初始化列表”不同	不写时编译器帮你拷贝；写了就要自己负责

模块	常见题型	核心规则	易错点	典型判断方法
成员初始化列表	<code>A(): y(x), x(1)</code>	成员初始化顺序按成员声明顺序, 不按初始化列表顺序	以为初始化列表书写顺序决定初始化顺序	看类中成员声明的先后
继承	public/protected/private 继承、访问控制	public 继承保持 public/protected; private 成员继承但派生类不能直接访问	“继承了”不等于“能直接访问”	先判断继承方式, 再判断成员原访问级别
多继承	菱形继承、二义性	非虚继承下, 多个基类对象会导致成员访问二义性	只看函数体里是否显式 <code>B::fun()</code> , 忽略数据成员是否二义	有多个路径继承同一基类时, 重点检查 <code>d.x</code> 、隐式转换
虚继承	<code>class B : virtual public A</code>	虚继承让公共基类对象只保留一份; 虚基类由最派生类负责构造	以为每条继承路径都有一份虚基类, 以为初始化列表能改变虚基类构造顺序	虚基类先构造, 顺序看最派生类的继承层次声明顺序
虚函数	基类指针/引用调用派生实现	虚函数一旦在基类声明为 <code>virtual</code> , 派生类同签名覆盖后仍是虚函数	构造函数不能是虚函数; 析构函数可以且常应为虚函数	看调用表达式的静态类型和函数是否 <code>virtual</code>
函数隐藏	派生类定义同名函数	派生类中的同名函数会隐藏基类所有同名重载	<code>B::F(double)</code> 会隐藏 <code>A::F(int)</code> , <code>b.F(2)</code> 可能调用 <code>B::F(double)</code>	名字查找先于重载决议; 要引入基类重载需 <code>using A::F;</code>
函数重载	参数类型、 <code>const</code> 、引用	只看函数名和参数列表; 返回值不能作为重载依据	误以为返回类型不同能重载	列候选函数, 排除不可行, 再比较匹配优先级
<code>const</code> 成员函数	<code>void f() const</code> 、 <code>const A a</code>	<code>const</code> 对象只能调用 <code>const</code> 成员函数和静态成员函数	静态成员函数没有 <code>this</code> , 可通过 <code>const</code> 对象调用	把成员函数隐含参数写成 <code>A* this</code> 或 <code>const A* this</code>
静态成员	<code>static int m;</code> 、 <code>static void f()</code>	静态数据成员属于类, 需要类外定义; 静态函数无 <code>this</code>	以为每个对象都有一份 <code>static</code> 数据成员	静态数据成员由所有对象共享
运算符重载	<code>[]</code> 、 <code>()</code> 、 <code>-></code> 、 <code><<</code> 、 <code>=</code> 、 <code>++</code>	<code>[]</code> 、 <code>()</code> 、 <code>-></code> 、 <code>=</code> 必须重载为成员函数; <code>::</code> 、 <code>.</code> 、 <code>.*</code> 、 <code>?:</code> 、 <code>sizeof</code> 不能重载	把 <code>operator[]</code> 写成普通友元函数	记住“必须成员”和“不可重载”两类清单
类型转换	转换构造、 <code>operator T()</code>	单参数构造可做隐式转换; 转换函数可把对象转成目标类型	<code>*s</code> 中对象可能先转成指针再解引用	先看是否有转换构造或类型转换运算符
异常	<code>try/catch</code> 、重抛、 <code>catch</code> 顺序	<code>catch</code> 参数像函数参数; <code>throw;</code> 重抛当前异常	<code>catch(...)</code> 应放最后; 重抛后不会继续执行 <code>try</code> 后普通语句	沿控制流跳到第一个匹配的 <code>catch</code>
模板	函数模板、类模板、显式特化	模板参数推导要求同一个 <code>T</code> 一致; 显式特化不是普通重载	<code>max(x,y,z)</code> 中 <code>x/y/z</code> 类型不一致会推导失败	对每个模板参数列出由实参推导出的类型
STL vector	<code>push_back</code> 、 <code>size</code> 、 <code>operator[]</code>	<code>v[i]</code> 不检查边界; 空 <code>vector</code> 的 <code>v[0]</code> 是越界未定义行为	误判为编译错误; 实际通常是运行时/未定义行为	<code>operator[]</code> 只要求类型可编译, 不保证元素存在

模块	常见题型	核心规则	易错点	典型判断方法
new/delete	动态对象、数组、构造析构	new 调构造，返回目标类型指针；malloc 不调构造，返回 void*	new[] 必须配 delete[]；不能用 delete 释放栈对象	看对象是单个还是数组、是否来自 new
输入输出	<<、指针、char*	cout << char* 按 C 字符串输出；其他指针通常输出地址	cout << *charPtr 输出单个字符，不是字符串	区分指针值、解引用值、字符串指针特例

函数调用优先级与重载决议

场景	优先级/规则	易错例子	正确理解
普通重载	精确匹配优先于提升，提升优先于标准转换，标准转换优先于用户自定义转换	f(3) 在 f(int) 和 f(double) 中选 f(int)	先列可行函数，再比较转换代价
const 对象调用成员函数	const 对象不能调用非 const 成员函数	const A a; a.f(3); 中 void f(int) 不可用	如果还有 static void f(double)，静态函数可用
派生类同名函数	派生类同名函数隐藏基类所有同名函数	B 定义 F(double) 后，A::F(int) 被隐藏	除非 B 中写 using A::F;
虚函数调用	通过基类指针/引用调用 virtual 函数时按动态类型	A* p = new B; p->f(); 调 B::f()	非 virtual 函数按静态类型绑定
默认参数和虚函数	默认参数看静态类型，虚函数体看动态类型	A* p = new B; p->foo(); 函数体可能是 B::foo，默认参数来自 A	默认实参看指针/引用静态类型，函数体如果 virtual 才看动态类型
构造/析构中调用虚函数	构造/析构期间虚调用不会跨越当前正在构造/析构的类层次	Base 构造/析构中调用 foo() 调 Base::foo(); Derived 构造/析构体中调用 foo() 调 Derived::foo()	不要把普通多态规则直接套到构造/析构函数内部
模板特化	显式特化不是普通重载	template<class T> f(T&); template<> f(const int&); int i; f(i); 输出主模板	i 是 int，主模板推导出 int& 更直接
转换构造与 explicit	直接初始化可用 explicit，拷贝初始化不可用 explicit	C c1(7) 调 C(int); C c2 = 7 不调 explicit C(int)	拷贝初始化会排除 explicit 构造
运算符表达式	运算符本质是函数调用，但保留求值语义和返回类型约束	a1[0] = a2[1] 调两个 operator[]	左侧需返回引用才能赋值

函数返回值类型：什么时候要加引用

目标	推荐返回类型	为什么	易错点
允许链式赋值	<code>A& operator=(const A&)</code>	返回当前对象本身，支持 <code>(a=b)=c</code> ，避免拷贝	返回 <code>A</code> 会多一次拷贝且语义弱
前置自增	<code>A& operator++()</code>	<code>++x</code> 返回自增后的原对象	返回局部变量引用会悬空
后置自增	<code>A operator++(int)</code>	<code>x++</code> 返回修改前的旧值，通常是值返回	不能返回局部对象引用
下标作为左值	<code>T& operator[](int)</code>	支持 <code>a[i] = value</code>	若返回 <code>T</code> ，只能读不能改原数组
const 下标只读	<code>T operator[](int) const</code> 或 <code>const T& operator[](int) const</code>	const 对象不能返回可修改引用	返回 <code>T&</code> 会破坏 const
输入输出流运算符	<code>ostream& operator<<(ostream&, const T&)</code>	支持 <code>cout << a << b</code>	返回 <code>ostream</code> 会拷贝流，通常不可行
返回对象内部成员	<code>T&</code> 或 <code>const T&</code>	需要修改内部成员或避免大对象拷贝时使用	不要返回局部变量引用
算术运算	<code>T operator+(...)</code> 、 <code>T operator-(...)</code>	结果是新对象	返回引用会指向临时或局部，错误
工厂函数/转换结果	<code>T</code>	产生新对象	不要为了省拷贝返回局部引用；现代 C++ 有返回值优化
链表/容器访问节点	指针或引用视所有权而定	指针表达“可能为空”，引用表达“必存在”	不清楚所有权时容易 double delete

运算符重载参数的 const 规则

场景	推荐写法	原因	例子
只读取右操作数	<code>const A& rhs</code>	避免拷贝，不允许修改实参	<code>A operator+(const A& rhs) const</code>
会修改右操作数	<code>A& rhs</code>	必须允许修改传入对象	少见，按题意需要才写
右操作数是小类型	<code>int x</code> 、 <code>double x</code>	内置小类型直接传值即可	<code>A operator+(int x) const</code>

场景	推荐写法	原因	例子
接收临时对象并转移资源	<code>A&& rhs</code>	移动语义	<code>A& operator=(A&& rhs) noexcept</code>
输出流左参数	<code>ostream& os</code>	输出会改变流状态，不能 <code>const</code>	<code>friend ostream& operator<<(ostream& os, const A& obj)</code>
输入流左参数	<code>istream& is</code>	输入会改变流状态和对对象	<code>friend istream& operator>> (istream& is, A& obj)</code>
成员函数不修改左操作数	函数末尾加 <code>const</code>	限制 <code>this</code> ，允许 <code>const</code> 对象调用	<code>bool operator<(const A& rhs) const</code>
赋值运算符	<code>A& operator= (const A& rhs)</code>	修改左操作数，返回左操作数引用	末尾不能加 <code>const</code>

口诀：

只读对象用 `const A&`；要改对象用 `A&`；小类型直接传值。
流对象不能 `const`；成员函数末尾 `const` 表示不改左操作数。

左值引用、右值引用和移动语义

写法	名称	能绑定什么	典型用途	易错点
<code>A&</code>	左值引用	有名字的对象，如 <code>A a;</code>	修改原对象、避免拷贝	不能绑定普通临时对象
<code>const A&</code>	<code>const</code> 左值引用	左值和右值都能绑定	只读传参、延长临时对象生命周期	不能通过引用修改对象
<code>A&&</code>	右值引用	临时对象、 <code>std::move(a)</code>	移动构造、移动赋值、接管资源	右值引用变量有名字后，表达式本身是左值
<code>A(const A&)</code>	拷贝构造	左值、 <code>const</code> 对象、临时对象	复制资源	源对象应保持原状态
<code>A(A&&) noexcept</code>	移动构造	右值、 <code>std::move</code> 后的对象	转移资源所有权	要把源对象置为可析构状态，如指针置 <code>nullptr</code>
<code>std::move(a)</code>	右值转换	把左值强制转成右值表达式	允许调用移动构造/移动赋值	<code>std::move</code> 本身不移动，只是类型转换
<code>noexcept</code>	不抛异常承诺	常用于移动构造/移动赋值	让 STL 容器更愿意移动元素	<code>noexcept</code> 函数若抛异常会 <code>terminate</code>

口诀：

A& 绑定普通对象；A&& 绑定临时对象或准备被搬走的对象。

拷贝构造复制资源，移动构造转移资源。

std::move 不移动，只是把对象转换成右值。

右值引用变量有名字，所以在表达式中仍然是左值。

* 的常见用法和判断

写法	含义	典型题点	易错点
T *p	声明指针变量	A *p = new B;	* 属于声明符，不是乘法
*p	解引用，取得指针指向的对象	cout << *s 中若 s 可转 char*，输出首字符	和 cout << p 输出指针/字符串区分
p->f()	等价于 (*p).f()	多态调用、成员访问	p 为空时解引用未定义
T& r = *p	引用绑定到指针所指对象	B &r = b0、基类引用绑定派生对象	引用不是新对象，不调用构造
delete p	删除单个动态对象	触发析构	基类指针删除派生对象要求基类析构 virtual
delete[] p	删除动态数组	依次析构数组元素	new[] 和 delete 混用是错误
int A::*pm	成员指针	早年卷可能出现	与普通指针不是一回事
operator*	重载解引用运算符	智能指针/迭代器类题	返回类型通常是 T&
char *s	C 字符串指针	cout << s 输出字符串	cout << *s 输出第一个字符
const int* p / int* const p	const 指针规则	const 在 * 左边：指向的内容不能改；const 在 * 右边：指针本身不能改	把“不能改内容”和“不能改指向”混淆

输出题通用解题流程

步骤	要做什么	对应考点	易错提醒
1	标出所有对象创建点	构造、拷贝构造、转换构造	A a = b; 是初始化，不是赋值

步骤	要做什么	对应考点	易错提醒
2	标出作用域和 <code>delete</code> 点	析构顺序	局部对象出作用域会自动析构
3	判断静态类型和动态类型	虚函数、多态	<code>virtual</code> 只影响通过指针/引用调用的虚函数
4	列出候选重载函数	重载、 <code>const</code> 、模板	先名字查找，再重载决议
5	检查是否有名字隐藏	继承中的同名函数	派生类同名函数隐藏基类同名重载
6	检查返回值是否为引用	<code>++</code> 、 <code>[]</code> 、赋值	返回引用会改变原对象，值返回只改临时
7	检查异常路径	<code>throw</code> 、 <code>catch</code> 、重抛	<code>throw</code> 后跳转， <code>try</code> 中后续语句不执行
8	最后处理输出格式	<code>endl</code> 、空格、无换行	析构函数里没 <code>endl</code> 就连续输出

函数题和程序填空题思路

题型	常见要求	解题模板	易错点
类接口补全	根据 judge 的调用写类	先从 <code>main</code> 和测试函数反推构造函数、运算符、类型转换	不要写多余输出；接口签名必须完全匹配
指针数组处理	<code>vec **arr</code> 、 <code>*val < *minv</code> 、 <code>static_cast<double>(*val)</code>	类中实现 <code>operator<</code> 、 <code>operator double</code> 、 <code>operator<<</code>	比较依据和平均值依据可能不是同一个成员
多态接口	<code>Shape* shape = new Rect;</code> <code>shape->area(); delete shape;</code>	基类写 <code>virtual double area() = 0;</code> <code>virtual ~Shape(){};</code>	忘记虚析构；派生类函数签名不匹配
链表填空	<code>Node* next</code> 、虚析构、 <code>AppendNode</code>	维护头尾指针；析构时先保存 <code>next</code> 再 <code>delete</code> 当前节点	<code>delete</code> 后再访问 <code>p->next</code> 是野指针
循环队列	<code>head</code> 、 <code>rear</code> 、 <code>queueSize</code>	空： <code>head == rear</code> ；满： <code>(rear+1)%size == head</code> ；数量： <code>(rear-head+size)%size</code>	队列容量是 <code>queueSize-1</code> ，不是 <code>queueSize</code>
分数类	<code>Fraction + - print</code>	<code>+/ -</code> 返回新对象；友元用于访问 <code>private</code> ；不要求约分则不要约分	全局运算符不用 <code>friend</code> 就不能访问私有成员
动态数组	<code>new[]</code> 、扩容、析构	构造分配，析构 <code>delete[]</code> ；拷贝要考虑深拷贝	默认拷贝会浅拷贝指针，导致 <code>double delete</code>
STL 字符串排序	多组输入、 <code>stop</code> 、稳定排序	读整行用 <code>getline</code> ；按长度稳定排序	混用 <code>cin >> n</code> 和 <code>getline</code> 要吃掉换行

题型	常见要求	解题模板	易错点
Map 前缀和	<code>insert</code> 覆盖、 <code>sum(prefix)</code>	可用 <code>map<string,int></code> 遍历前缀	插入已有键要覆盖，不是累加
模板函数补全	<code>template<class ...></code> 、迭代器	从函数调用反推模板参数和操作符参数	<code>++first1</code> 后别忘 <code>++first2</code>

异常对象生命周期和拷贝次数

场景	会发生什么	输出题怎么数
<code>throw</code> 临时对象	直接构造异常对象，现代 C++ 可能有拷贝消除	重点看题目是否有构造/拷贝构造输出，老题常按“构造异常对象”算
<code>throw</code> 局部对象	通常会从局部对象拷贝构造出异常对象；随后局部对象在栈展开时析构	先输出局部对象构造，再输出异常对象拷贝构造，再析构局部对象
栈展开	<code>throw</code> 后离开当前作用域，已经构造完成的局部对象按逆序析构	<code>try</code> 中 <code>throw</code> 后面的语句不执行， <code>throw</code> 前已经构造的对象要析构
<code>catch(Exception e)</code>	按值捕获，会从异常对象再拷贝构造 <code>catch</code> 参数 <code>e</code>	多一次拷贝构造； <code>catch</code> 结束时先析构 <code>e</code>
<code>catch(Exception& e)</code>	按引用捕获，不拷贝异常对象	没有 <code>catch</code> 参数拷贝；更推荐
<code>catch(const Exception& e)</code>	按 <code>const</code> 引用捕获，不拷贝，且不修改异常对象	最常见写法
<code>catch(...)</code>	捕获任意异常，不知道具体类型	不会构造具名 <code>catch</code> 参数
<code>catch</code> 结束	<code>catch</code> 参数先析构，最后异常对象本体销毁	按值捕获时通常看到两个析构： <code>catch</code> 参数、异常对象本体
<code>throw;</code>	在 <code>catch</code> 中重抛当前异常对象	不重新构造异常对象，只继续传播当前异常
<code>throw e;</code>	抛出表达式 <code>e</code> ，可能重新拷贝构造新的异常对象	和 <code>throw;</code> 不同，输出题要多算拷贝

分析口诀：

`throw` 局部对象：局部对象 -> 异常对象 -> 栈展开析构局部对象。

`catch` 按值：再拷贝一份 `catch` 参数。

`catch` 按引用：不拷贝。

`throw` 后 `try` 里后续语句不执行。

`throw;` 是重抛当前异常，`throw e;` 是重新抛出 `e`。

模板类专项

考点	标准写法	常见题目	易错点
模板类声明	<pre>template <class T> class Array { ... };</pre>	动态数组、队列、向量、节点类	漏写 <code>template<class T></code>
类外成员函数定义	<pre>template<class T> T& Array<T>::operator[](int i)</pre>	程序填空补 <code>operator[]</code> 、 <code>iterate</code>	写成 <code>Array::operator[]</code> ，漏掉 <code><T></code>
模板构造函数	<pre>template<class T> CQueue<T>::CQueue(int s) : queueSize(s) { ... }</pre>	循环队列模板类	<code>const</code> 数据成员必须在初始化列表中初始化
模板析构函数	<pre>template<class T> CQueue<T>::~~CQueue(){ delete[] data; }</pre>	动态内存模板类	<code>new[]</code> 对应 <code>delete[]</code>
模板类指针成员	<pre>Array<T>* next;</pre>	分块数组、链表	写成 <code>Array* next</code> 在模板类外通常不完整/不规范
模板类对象声明	<pre>Array<int> a;、CQueue<double> rq;</pre>	主函数填空	漏写类型参数
模板类静态成员	<pre>template<class T> int A<T>::cnt = 0;</pre>	统计对象个数	每个 <code>T</code> 都有自己的一份 <code>static</code> 成员
模板类运算符	<pre>template<class T> T& Array<T>::operator[](int i)</pre>	<code>a[i] = i</code>	左值下标必须返回引用
模板类友元输出	<pre>friend ostream& operator<< <T> (ostream&, const Vec<T>&); 或类 内定义 friend</pre>	输出模板对象	友元模板语法复杂，考试可优先类内直接定义
模板特化	<pre>template<> class A<int> { ... };</pre>	<code>A<double></code> 、 <code>A<char></code> 、 <code>A<int></code> 输出	特化版本完全替代主模板对应类型

考点	标准写法	常见题目	易错点
模板参数推导	函数模板由实参推导，类模板通常要显式写类型	<code>inner_product</code> 、 <code>max</code>	同一个 <code>T</code> 不能同时由 <code>int</code> 和 <code>double</code> 推出
模板和继承	<code>class Disk : public Shape</code> 通常不是模板； <code>CQueue<T></code> 才是模板	类设计题区分普通多态和模板容器	不要把所有类都写成模板，只有数据类型可变才模板化

模板类程序填空固定套路

位置	先想什么	常见答案形态
类名前一行	这是不是模板类	<code>template <typename T></code>
成员数据类型	数据是否随模板参数变化	<code>T *data;</code> 、 <code>T value;</code> 、 <code>Node<T>* next;</code>
类外函数名前	是否需要模板头	<code>template <typename T></code>
作用域限定	类名是否要带模板参数	<code>Array<T>::</code> 、 <code>CQueue<T>::</code>
返回类型	是否需要作为左值或链式调用	<code>T&</code> 、 <code>Class<T>&</code>
构造函数	是否有 <code>const</code> 成员或动态内存	初始化列表、 <code>new T[size]</code>
析构函数	是否有 <code>new</code> 或 <code>new[]</code>	<code>delete</code> 、 <code>delete[]</code>

重要专题速查

专题	必背结论	例子
不可重载运算符	<code>::</code> 、 <code>.</code> 、 <code>.*</code> 、 <code>?:</code> 、 <code>sizeof</code>	选择题常问 “which cannot be overloaded”
必须作为成员重载	<code>operator=</code> 、 <code>operator[]</code> 、 <code>operator()</code> 、 <code>operator-></code>	<code>[]</code> 不能写成 <code>friend</code> 全局函数
构造/析构能否 virtual	构造函数不能 <code>virtual</code> ；析构函数可以 <code>virtual</code>	多态基类通常要 <code>virtual</code> 析构
析构能否重载	析构函数不能重载，一个类最多一个析构函数	<code>~A()</code> 无参数，无返回值
<code>new</code> vs <code>malloc</code>	<code>new</code> 是运算符，调用构造，返回类型化指针； <code>malloc</code> 是函数，不调用构造，返回 <code>void*</code>	OOP 选择题高频
<code>delete</code> 安全性	<code>delete nullptr</code> 安全；重复 <code>delete</code> 同一指针不安全	“It is safe to delete the same pointer multiple times” 是错
<code>vector::operator[]</code>	不做边界检查	空 <code>vector</code> 的 <code>v[0] = 2.5</code> 是未定义行为
<code>const</code> 数据成员	必须在构造函数初始化列表中初始化	不能在构造函数体内赋值

专题	必背结论	例子
static 数据成员	类内声明，类外定义；所有对象共享	<code>int A::m;</code>
同类对象访问 private	类的成员函数可以访问同类任意对象的 private 成员	<code>int fun(A& another){ another.x += x; }</code> 合法
namespace	同名函数可用命名空间区分	<code>A::func(a)</code> 、 <code>B::func(b)</code>
inline	主要减少函数调用开销，不是减少代码长度	判断题常设反向陷阱

针对历年卷的复习优先级

优先级	内容	原因	建议
最高	构造/拷贝/赋值/析构输出顺序	几乎每年都出现输出题	每题先画对象生命周期
最高	函数重载、隐藏、const、模板推导	选择题和输出题高频，细节多	严格按“名字查找 -> 可行函数 -> 最佳匹配”
最高	运算符重载返回类型	程序填空和函数题常考	<code>=</code> 、 <code>++前置</code> 、 <code>[]</code> 常返回引用； <code>+/-</code> 常返回值
高	虚函数、多态、虚析构	形状面积、文档图形、Base/Derived 输出题常见	基类接口写纯虚函数和虚析构
高	指针、引用、动态内存	链表、队列、数组、对象切片都依赖	明确所有权：谁 new，谁 delete
高	异常处理	2018、2023 都出现	记住 catch 顺序、重抛、异常后控制流
中	STL 容器和算法	vector、map、string 排序题	注意 <code>getline</code> 、稳定排序、边界
中	模板类/模板函数	inner_product、Array、vec 类题	从调用点反推接口，不要过度设计
中	namespace、static、friend	选择/填空常考	friend 破访问控制，不是成员函数

期末冲刺清单

顺序	复习内容	必做题型	检查点
1	构造、拷贝构造、赋值、析构	程序输出题	能区分 <code>A a=b</code> 、 <code>a=b</code> 、函数传参、返回对象

顺序	复习内容	必做题型	检查点
2	继承、多态、virtual、对象切片	程序输出题、选择题	能判断调用基类还是派生类函数，析构是否完整
3	函数重载、隐藏、const、static	程序输出题、选择题	能解释 <code>b.F(2)</code> 、 <code>const A a; a.f(3)</code> 这类题
4	运算符重载和返回引用	填空题、函数题	<code>operator=</code> 、 <code>operator[]</code> 、 <code>operator++</code> 、 <code>operator<<</code> 返回类型不出错
5	模板类和模板函数	填空题、函数/类设计题	类外定义能写对 <code>template<class T></code> 和 <code>Class<T>::</code>
6	exception 和 casting	选择题、输出题	<code>catch</code> 顺序、 <code>throw;</code> 、 <code>dynamic_cast</code> 条件能判断
7	函数/类设计题	大题	先从 judge 代码反推接口，再写实现，不改测试代码

继承和生命周期易错补充

易错点	正确理解	例子
<code>X can be assigned to Y</code> 的方向	英文意思是“X 可以赋值给 Y”，代码方向是 <code>Y = X</code>	<code>Derived can be assigned to Base</code> 表示 <code>Base b; Derived d; b = d;</code>
派生对象赋给基类对象	可以，但发生对象切片，只保留基类子对象部分	<code>Base b = derived;</code> 后 <code>b</code> 仍然是 <code>Base</code> 对象
基类对象赋给派生对象	默认不可以，除非派生类自定义 <code>operator=(const Base&)</code> 或转换	<code>Derived d; Base b; d = b;</code> 通常编译错误
派生类指针赋给基类指针	可以，向上转型安全	<code>Base* p = new Derived;</code>
基类指针赋给派生类指针	默认不可以；需要 <code>dynamic_cast</code> 并检查结果	<code>Derived* p = dynamic_cast<Derived*>(basePtr);</code>
基类指针删除派生对象	基类析构函数必须是 <code>virtual</code> ，否则未定义行为	<code>Base* p = new Derived; delete p;</code> 要求 <code>virtual ~Base()</code>
成员对象生命周期	成员对象先于所属类构造函数体构造，晚于所属类析构函数体析构	<code>Base</code> 中有 <code>MyClass m;</code> ：先 <code>MyClass::Ctor</code> ，再 <code>Base::Ctor</code> ；先 <code>Base::Dtor</code> ，再 <code>MyClass::Dtor</code>
派生对象完整构造顺序	基类子对象 -> 成员对象 -> 派生类构造函数体	<code>Derived(int): Base(...), member(...) {}</code>

易错点	正确理解	例子
虚基类构造顺序	虚基类先于普通基类构造；顺序看最派生类的继承层次声明顺序，不看构造函数初始化列表	<code>class C : virtual B, virtual A</code> 中若 <code>B</code> 又虚继承 <code>A</code> ，构造 <code>C</code> 时通常先构造共享的 <code>A</code> ，再构造 <code>B</code>
自定义派生类拷贝构造	初始化列表不写基类/成员时，它们会默认构造；要拷贝必须显式写	<code>B(const B& b): A(b), a(b.a), i(b.i) {}</code>
未自定义派生类拷贝构造	编译器生成的版本会自动拷贝构造基类和成员对象	相当于 <code>B(const B& other): A(other), member(other.member), ... {}</code>
派生对象完整析构顺序	派生类析构函数体 -> 派生成员 -> 基类析构函数体 -> 基类成员	与构造顺序反向
构造/析构中的虚函数	构造/析构函数内部调用虚函数时，不会调用“尚未构造完成”或“已经析构掉”的派生层实现	<code>Base</code> 构造中 <code>foo()</code> 调 <code>Base::foo()</code> ； <code>Derived</code> 析构体中 <code>foo()</code> 调 <code>Derived::foo()</code>

最容易丢分的细节清单

细节	正确写法/结论	错误写法/误区
拷贝构造参数	<code>A(const A& rhs)</code>	<code>A(A& rhs)</code> 适用范围太窄
派生类初始化基类	<code>B(int x): A(x) {}</code>	以为只能由编译器调用基类默认构造，或在派生类初始化列表直接写基类成员名
派生类拷贝构造	<code>B(const B& b): A(b), a(b.a), i(b.i) {}</code>	自定义 <code>B(const B&)</code> 后，以为基类和成员对象会自动拷贝构造
默认派生类拷贝构造	不自定义 <code>B(const B&)</code> 时，编译器会自动拷贝基类和成员对象	把它和“自定义但没写初始化列表”的行为混为一谈
赋值返回	<code>A& operator=(const A& rhs)</code>	返回 <code>void</code> 或 <code>A</code> 影响链式赋值
前置 ++	<code>A& operator++()</code>	返回值会导致无法连续修改原对象
后置 ++	<code>A operator++(int)</code>	返回引用到局部旧值
下标左值	<code>T& operator[](int)</code>	返回 <code>T</code> 导致 <code>a[i]=x</code> 不能改原对象
const 下标	<code>T operator[](int) const</code> 或 <code>const T&</code>	返回非 <code>const</code> 引用破坏只读性
输出运算符	<code>friend ostream& operator<<(ostream&, const A&)</code>	返回 <code>ostream</code> 或漏掉引用
<code>operator[]</code>	必须成员函数	写成非成员 <code>friend</code>

细节	正确写法/结论	错误写法/误区
运算符参数 const	只读右操作数通常写 <code>const A&</code> ，成员函数末尾 <code>const</code> 表示不改左操作数	把所有参数都写成 <code>const</code> ，导致 <code>istream&</code> 或需要修改的参数无法工作
基类析构	<code>virtual ~Base(){} </code>	多态删除时基类析构非 <code>virtual</code>
基类指针删除派生对象	<code>Base* p = new Derived; delete p;</code> 要求 <code>Base</code> 析构为 <code>virtual</code>	以为类里有其他虚函数就一定能安全删除；析构函数本身也应 <code>virtual</code>
成员对象生命周期	成员先构造，所属类构造函数体后执行；析构时所属类析构函数体先执行，成员后析构	把成员对象当成在构造函数体里才开始构造
虚基类构造	虚基类构造顺序看最派生类的继承层次声明顺序，不看构造函数初始化列表	以为 <code>C(): B(), A()</code> 可以改变虚基类构造顺序
assigned to	<code>X assigned to Y</code> 对应代码 <code>Y = X</code>	把赋值方向理解反
动态数组释放	<code>delete[] p</code>	<code>delete p</code>
链表析构	先保存 <code>next</code> ，再 <code>delete</code> 当前节点	<code>delete</code> 后访问当前节点成员
模板参数一致	<code>T</code> 由多个实参推导必须一致	<code>max(int, int, float)</code> 直接推导同一 <code>T</code> 会失败
<code>char*</code> 输出	<code>cout << s</code> 输出字符串， <code>cout << *s</code> 输出首字符	把 <code>*s</code> 当字符串
<code>const</code> 成员	函数尾部 <code>const</code> 限制 <code>this</code>	以为只限制返回值
对象切片	<code>A a = b;</code> 只保留 <code>A</code> 子对象	以为还能虚调用 <code>B</code> 行为
默认参数 + <code>virtual</code>	默认参数看静态类型，虚函数体看动态类型	以为虚函数的默认参数也会跟着动态绑定
<code>const</code> 指针	<code>const</code> 在 <code>*</code> 左边：指向的内容不能改； <code>const</code> 在 <code>*</code> 右边：指针本身不能改	把 <code>const int* p</code> 和 <code>int* const p</code> 看成一样

```
//Matrix.h
#ifndef MATRIX_H
#define MATRIX_H

template<typename T>
class Matrix{
private:
    int row,col;
    T* data;
public:
    Matrix(int m = 0 ,int n = 0):row(m),col(n){
        if(m*n != 0){
            data = new T[m*n];
        }else data = NULL;
    }
};
```

```

}
~Matrix(){
    delete[] data;
}

friend istream& operator>> <>(istream& in, Matrix& obj){
    for(int i = 0; i<obj.row; i++){
        for(int j = 0; j<obj.col; j++){
            in >> obj.data[i*obj.col + j];
        }
    }
    return in;
}

Matrix& operator=(const Matrix& other) {
    if (this == &other)
        return *this;

    delete[] data;

    row = other.row;
    col = other.col;

    int size = row * col;
    data = size ? new T[size] : NULL;
    for (int i = 0; i < size; i++)
        data[i] = other.data[i];

    return *this;
}

friend ostream& operator<< <>(ostream& out, Matrix& obj){
    for(int i = 0; i<obj.row; i++){
        for(int j = 0; j<obj.col; j++){
            out << obj.data[i*obj.col + j]<<" ";
        }
        out << endl;
    }
    return out;
}

Matrix& operator+ <>(const Matrix& rhs){
    for (int i = 0; i < row * col; i++)
        data[i] = data[i] + rhs.data[i];
    return *this;
}
}
#endif

```

